

08/4/26

Assignment - 01

1. With help of a neat flow diagram, explain the Algorithm design and analysis process.

Ans :-

Understand the problem

Decide on : Computational means, exact vs approximate solving algorithm design techniques

Design an algorithm

Prove correctness

Analyze the algorithm.

Code the algorithm.

1) Understanding the problem : Read the problem description carefully to understand the problem statement completely.

2) Ask questions for clarifying doubts about the problem.

3) Identify the problem type and use the existing algorithms to find solution.

4) Input (Instances) to the problem and range of the input gets fixed.

5) Decision-making:-

a) Choice of computational devices like processor, memory is mainly based on space and time efficiency.

b) Choosing b/w exact vs approximate problem solving.

6) Algorithm Design techniques:-

a) Algorithm + Data structure = Program

b) Methods of specifying an algorithm

1) Natural Language

2) Flow Chart

3) Pseudocode

7) Proving an algorithm correctness.

8) Analyzing an algorithm:-

* Time efficiency - indicating how fast an algorithm runs.

* Space efficiency - indicating how much extra memory it uses.

9) Coding an algorithm.

2) Consider the following:-

i) Algorithm GUESS($A[i][j]$)

for $i \leftarrow 0$ to $n-1$

for $j \leftarrow 0$ to i

$A[i][j] \leftarrow 0$

→ What does the algorithm compute?

It fills the lower triangular part (including diagonal) of the matrix with 0.

→ What is the basic operation?

This assignment happens multiple times. $[A[i][j] \leftarrow 0]$.

→ What is the efficiency of this algorithm?

Time complexity = $O(n^2)$

ii) Algorithm Enigma(n, a)

// Input: An array of $a[0 \dots n-1, 0 \dots n-1]$ of real no

for $i \leftarrow 0$ to $n-2$ do

for $j \leftarrow i+1$ to $n-1$ do

if $(a[i, j] \neq a[j, i])$ return false

return true

→ What does the algorithm compute?

The algorithm checks if a square matrix is

symmetric.

→ What is the basic operation?
The basic comp operation is the comparison of two matrix elements.

→ What is the efficiency of the algorithm?

Time efficiency: $C(n) = O(n^2)$

3. Solve the following recurrence relations.

i) $x(n) = 3x(n-1)$ for $n > 1, x(1) = 4$.

$$\text{Sol: } x(n) = \begin{cases} 3x(n-1) & , n > 1 \\ 4 & , n = 1 \end{cases}$$

$$\begin{aligned} x(n) &= 3x(n-1) \\ &= 3[3x(n-2)] \\ &= 3^2 x(n-2) \\ &= 3^2 [3x(n-3)] \\ &= 3^3 x(n-3) \\ &= 3^3 [3x(n-4)] \\ &= 3^4 x(n-4) \\ &\vdots \\ &= 3^i x(n-i) \end{aligned}$$

$$\begin{aligned} i &= n-1 \\ &= 3^{n-1} x(n-(n-1)) \\ &= 3^{n-1} x(1) \\ &= 3^{n-1} \cdot 4 \\ &= 3^{n-1} \cdot 4 \\ &= \frac{3^n}{3} \cdot 4 \\ &= 3^{n-1} \cdot 4 \\ &= 3^n \cdot 4/3 \end{aligned}$$

$$\boxed{x(n) = O(3^n)}$$

$$\begin{aligned} x(n) &= 3x(n-1) \quad \text{--- (1)} \\ n &= n-1 \\ x(n-1) &= 3x(n-1)-1 \\ &= 3x(n-2) \\ n &= n-2 \\ x(n-2) &= 3x(n-2)-1 \\ &= 3x(n-3) \\ n &= n-3 \\ x(n-3) &= 3x(n-3)-1 \\ &= 3x(n-4) \end{aligned}$$

ii) $x(n) = x(n/2) + n$ for $n > 1, x(1) = 1$

$$\text{Sol: } x(n) = \begin{cases} x(n/2) + n & , n > 1 \\ 1 & , n = 1 \end{cases}$$

$$= x(n/8) + n/4 + n/2 + n$$

$$x(n) = n + n/2 + n/4 + \dots + 1$$

$$x(n) = 2n - 1$$

$$\boxed{C(n) = O(n)}$$

iii) $M(n) = 2M(n-1) + 1; M(1) = 1$

$$M(n) = \begin{cases} 2M(n-1) & , n > 1 \\ 1 & , n = 1 \end{cases}$$

$$\begin{aligned} M(n) &= 2M(n-1) \\ &= 2(2M(n-2)) \\ &= 2^2 M(n-2) \\ &= 2(2M(n-3)) \\ &= 2^3 M(n-3) \end{aligned}$$

$$\begin{aligned} m(n) &= 2M(n-1) \quad \text{--- (1)} \\ n &= n-1 \\ M(n-1) &= 2M(n-1)-1 \\ &= 2M(n-2) \\ n &= n-2 \\ M(n-2) &= 2M(n-2)-1 \\ &= 2M(n-3) \end{aligned}$$

$$\begin{aligned} &= 2^i M(n-i) \\ &> i = n-1 \\ &= 2^i M(n-(n-1)) \\ &= 2^{n-1} M(1) \\ &= 2^{n-1} \cdot 1 \\ &= \frac{2^n}{2} \end{aligned}$$

$$\boxed{C(n) = 2^n}$$

iv) $X(n) = x(n-1) + 5$ for $n > 1$, $x(1) = 0$

Sol:- $X(n) = \begin{cases} x(n-1) + 5 & n > 1 \\ 1 & n = 0 \end{cases}$

$$X(n) = x(n-1) + 5$$

$$= x(n-2) + 5 + 5$$

$$= x(n-2) + 10$$

$$= x(n-3) + 10 + 5$$

$$= x(n-3) + 15$$

$$\vdots$$

$$= x(n-i) + 5$$

$$\text{①} \Rightarrow i = n+1$$

$$= x(n+1)$$

$$x(n) = 5(n-1)$$

$$x(n) = 5n - n$$

$$\therefore T(n) = O(n)$$

4) System needs to sort employee records where:

- Swapping large records is very costly.
- Comparisons are inexpensive

The method used:

- Selects the smallest element from the unsorted portion
- Places it in the correct position.

• Repeats from remaining elements.

Tasks:

1. Apply the method
2. Show each step
3. Count swaps and comparisons
4. Analyze suitability

Given: $A = \{29, 10, 14, 37, 13\}$

Ans:- We can apply Selection sort to this problem.

0	1	2	3	4
29	10	14	37	13

for $i = 0$ to $n-2$ do
 $\min = i$
 for $j = i+1$ to $n-1$ do
 if $A[j] < A[\min] \Rightarrow \min = j$
 swap $A[i]$ and $A[\min]$

$i = 0$ to $5-2(3)$ do

$\min = 0$

$j = 0+1$ to 4 do

$A[1] < A[0]$

$10 < 29 \checkmark$

$\min = 1$

$j = 2$ to 4

$A[2] < A[1]$

$14 < 10 \checkmark$

$j = 3$ to 4

$A[3] < A[1]$

$27 < 10 \times$

$j = 4$ to 4

$A[4] < A[1]$

$13 < 10 \times$

Swap $A[0]$ & $A[1]$

$A[0] < A[1]$

$10 < 29 \checkmark$

10	29	14	37	13
----	----	----	----	----

$i = 1$ to 5 do

$\min = i$

$j = 2$ to 4 do

$A[2] < A[1]$

$14 < 29 \checkmark$

$\min = 2$

$j = 3$

$A[3] < A[2]$

$37 < 14 \times$

$j = 4$

$A[4] < A[2]$ obt min =

$13 < 14 \checkmark$

Swap $A[1]$ & $A[4]$

0, 29, 13, 4

10 | 13 | 14 | 37 | 29

$i = 2$ to 3 do

$\min = 2$

$j = 3$ to 4 do

$A[3] < A[2]$

$37 < 14 \times$

$\min = 2$

$j = 4$ to 4

$A[4] < A[2]$

$29 < 14 \times$

$\min = 2$

$i = 3$ to 3 do

$\min = 3$

$j = 4$ to 4 do

$A[4] < A[3]$

$29 < 37 \checkmark$

obt $\min = 4$

Swap $A[3]$ & $A[4]$

37 & 29

10 | 13 | 14 | 37 |

10 | 13 | 14 | 29 | 37

Time complexity $C(n) = O(n^2)$

5) A small e-learning platform stores student quiz scores in an array. The data is almost sorted, but occasionally a few scores are misplaced due to late updates.

The system uses a method where:

- Adjacent elements are compared
- Larger elements gradually move to the end in each pass
- The process stops early if no swaps occur

Tasks:

- Apply the given method to sort the data
- Show all passes

- Analyze its performance for this scenario
- Justify whether it is suitable

Given:

$A = \{50, 60, 70, 65, 80\}$

Sol:- We can apply Bubble sort as it sorts by comparing adjacent element in list and exchange if not in order & keep repeating.

$A = 50 | 60 | 70 | 65 | 80$

for $i = 0$ to $n-2$ do

for $j = 0$ to $n-2-i$ do

if $A[j+1] < A[j]$

Swap $A[j]$ & $A[j+1]$

$i = 0$ to 3

$j = 0$ to 3 do

$j = 0$ $A[0+1] < A[0]$

$60 < 50 (F)$

$j = 1$ $A[1+1] < A[1]$

$70 < 60 (F)$

$j = 2$ $A[2+1] < A[2]$

$65 < 70 (T)$

Swap $A[2]$ & $A[3]$

50 | 60 | 65 | 70 | 80

Time complexity = $C(n) = O(n^2)$
Bubble sort is suitable as we compare every element with its adjacent element & we swap only if not in order. by this larger elements gradually move to the end in each pass.

6. A digital library system stores a large collection of book records that must be arranged based on publication year. The system must ensure that:

- The sorting process is efficient for handling large volumes of data.

- Records with the same publication year maintain their original order.

To achieve this, the system uses a technique where:

- The dataset is repeatedly divided into smaller sublists.
- Each subject is processed independently.
- The sorted subjects are then combined to produce to final ordered list.

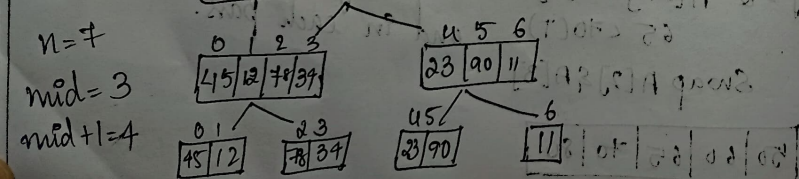
Task: 1. Apply the given method to sort the dataset.

- Illustrate the step-by-step division of the array.
- Show how the sublists are merged to obtain the final result.
- Draw the recursion structure (tree star representation).
- Analyze the time & space complexity.

$A = \{45, 12, 78, 34, 23, 90, 11\}$

Sol:- A

45	12	78	34	23	90	11
----	----	----	----	----	----	----



1/2, 4/5

12	45	78	34
----	----	----	----

12	34	45	78
----	----	----	----

23	11	23	90
----	----	----	----

11	12	23	34	45	78	90
----	----	----	----	----	----	----

Time complexity: $O(n \log n)$

Space complexity: $O(n)$

7. Solve the recurrence relation:

$\Rightarrow M(n) = 4M(n/2) + n^2$ for $n > 1$; $M(1) = 1$

Sol:-

$$M(n) = 4M(n/2) + n^2$$

$$M(n) = 4\left(4M\left(\frac{2n}{2}\right) + \frac{n^2}{4}\right) + n^2$$

$$= 4^2 \left[M\left(\frac{2n}{2} + \frac{n^2}{4}\right) \right] + n^2$$

$$= 4^3 \left[\frac{2n}{2} + \frac{2n + n^2}{4} \right] + n^2 + n^2$$

$$M(n) = 4M(n/2) + n^2 \quad \text{--- (1)}$$

$$M(n/2) = 4M\left(\frac{n}{2}\right) + \left(\frac{n}{2}\right)^2$$

$$= 4M\left(\frac{2n}{2}\right) + \frac{n^2}{4}$$

$$M(n/4) = 4M\left(\frac{n}{4}\right) + \frac{n^2}{16}$$

$$= 4M\left(\frac{2n}{4}\right) + \frac{n^2}{16}$$

$$\therefore M(n) = O(n^2 \log n)$$

$$C(n) = n \log n$$

ii) $M(n) = M(n-1) + 3$ for $n > 0$; $M(1) = 1$

$$M(n) = \begin{cases} M(n-1) + 3, & n > 0 \\ 1, & n = 1 \end{cases}$$

$$M(n) = M(n-1) + 3$$

$$= M[n-2] + 3 + 3$$

$$= M[n-3] + 3 + 3 + 3$$

$$= M[n-i] + i$$

$$= M[n - (n-1)] + i$$

$$= M(1) + i$$

$$= 1 + (n-1)$$

$$M(n) = n$$

$$\therefore C(n) = O(n)$$

iii) $T(n) = 2T(n-1) + 5$ for $n > 0$; $T(1) = 3$

$$T(n) = \begin{cases} 2T(n-1) + 5, & n > 0 \\ 3, & n = 1 \end{cases}$$

$$T(n) = 2T(n-1) + 5$$

$$T(n) = 2T(n-1) + 5 - \text{①}$$

$$\begin{aligned} n &= n-1 \\ T(n-1) &= 2T(n-2) + 5 \end{aligned}$$

$$= 2(2T(n-2)) + 5 + 5$$

$$= 2^2 T(n-2) + 10$$

$$= 2^2 [2T(n-3)] + 10 + 5$$

$$= 2^3 T(n-3) + 15$$

$$= 2^i T(n-i) + i$$

$$i = n-1$$

$$= 2^i T(n - (n-1)) + i$$

$$= 2^i T(1) + i$$

$$= 2^i T(1) + i$$

$$= 2^i \times 3 + i$$

$$= 2^{n-1} \times 3 + (n-1)$$

$$= \frac{2^n}{2^1} \times 3 + (n-1) = 2^n \times \frac{3}{2} + (n-1)$$

$$\therefore C(n) = O(2^n)$$

8. A data analysis system processes a list of transaction values that must be arranged in ascending order efficiently. The system follows a strategy where:

- A specific element is chosen as a reference.
- The dataset is rearranged such that:
 - ↳ Elements smaller than the reference are placed on the one side.
 - ↳ Elements larger than the reference are placed on other side.
- The same process is recursively applied to the resulting sublists.

Tasks: 1. Apply the given steps clearly
 2. Show all partition steps clearly
 3. Draw the recursive structure
 4. Analyze the time complexity
 5. Suggest an improved strategy

Sol:- $A = \{45, 12, 78, 34, 23, 90, 11\}$

$\boxed{45 | 12 | 78 | 34 | 23 | 90 | 11}$

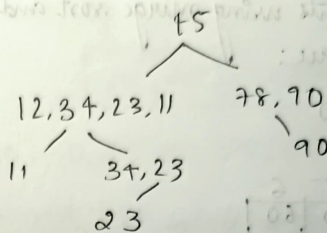
pivot

$\{12, 34, 23, 11\} : 45 : \{78, 90\}$

pivot

$\{11\} | 12 | \{23\} | 34 : 78 | 90$

The final sorted array is $\{11, 12, 23, 34, 45, 78, 90\}$



∴ Time complexity $(C(n)) = O(n^2)$

9. Compare Merge sort and Quick sort based on:

↳ Time complexity

↳ Space complexity

↳ Stability

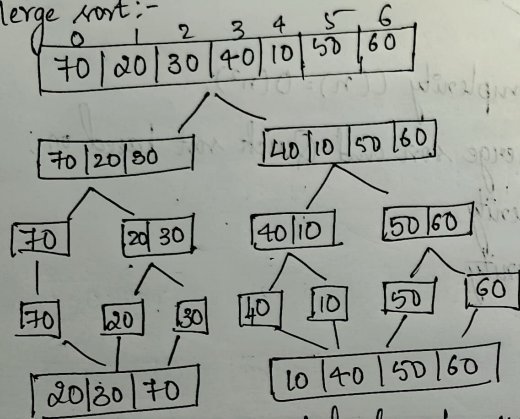
Ans:-

Feature	Quick Sort	Merge sort
Time complexity	Worst: $O(n^2)$	Worst: $O(n \log n)$
Space complexity	$O(\log n)$	$O(n)$
Stability	Not Stable	Stable

10. Sort the following elements using merge sort and quick sort. Write the recursion tree:

1) 70, 20, 30, 40, 10, 50, 60

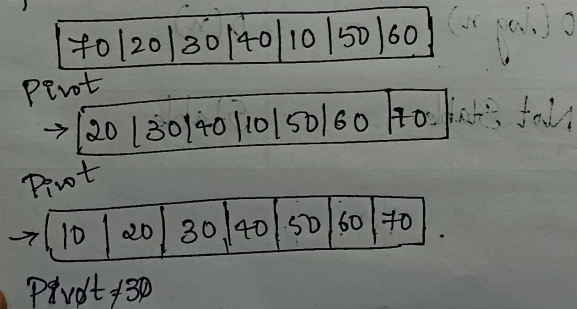
Merge sort:-



The final sorted elements are

10	20	30	40	50	60	70
----	----	----	----	----	----	----

Quick sort:-

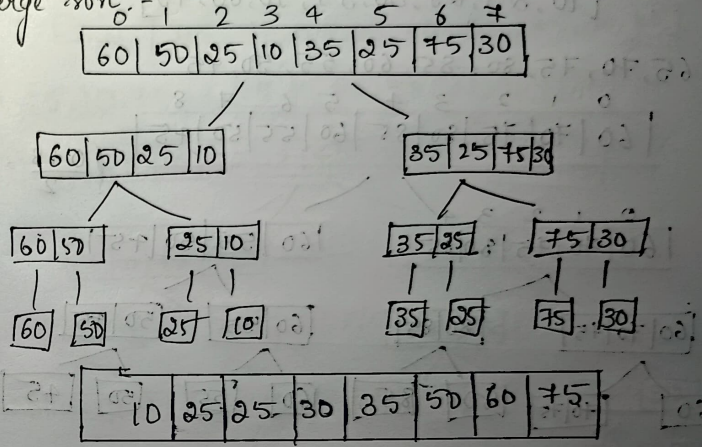


∴ The final sort is

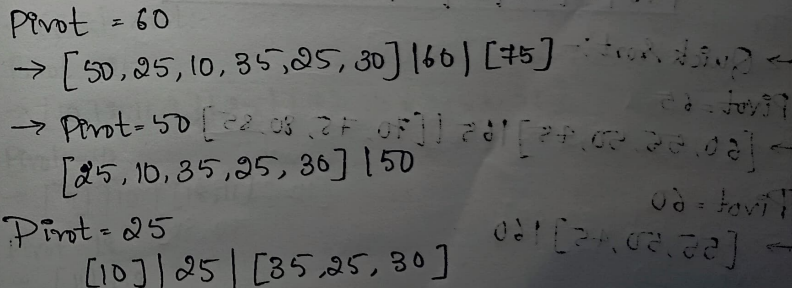
10	20	30	40	50	60	70
----	----	----	----	----	----	----

2) 60, 50, 25, 10, 35, 25, 75, 30

Merge sort:-



Quick sort:-



Pivot = 35

→ [25, 30] | 35

Pivot = 25

→ 25 | 30

∴ [10, 25, 25, 30, 35, 50, 60, 75]

3> 65, 70, 75, 80, 85, 60, 55, 50, 45

0	1	2	3	4	5	6	7	8
60	70	75	80	85	60	55	50	45

$\frac{0+8}{2} = 4$

0	1	2	3	4
60	70	75	80	85

60	55	50	45
----	----	----	----

60	70	75
----	----	----

80	85
----	----

60	55
----	----

50	45
----	----

60	70	75	80	85	60	55	50	45
----	----	----	----	----	----	----	----	----

45	50	55	60	60	70	75	80	85
----	----	----	----	----	----	----	----	----

→ Quick sort:- [25, 30] | [10, 25, 25, 30, 35, 50, 60, 75]

Pivot = 65

→ [60, 55, 50, 45] | 65 | [70, 75, 80, 85]

Pivot = 60

→ [55, 50, 45] | 60

[10, 25, 25, 30, 35, 50, 60, 75]

→ Pivot = 55

→ [50, 45] | 55

Pivot = 50

→ [45] | 50

Right side: Pivot = 70

→ 70 | [75, 80, 85]

Pivot = 75

→ [80, 85]

Pivot = 80

→ [85]

∴ Final [45, 50, 55, 60, 65, 70, 75, 80, 85]

4> 25, 10, 72, 18, 40, 11, 64, 58, 32, 9

0	1	2	3	4	5	6	7	8	9
25	10	72	18	40	11	64	58	32	9

25	10	72	18	40
----	----	----	----	----

11	64	58	32	9
----	----	----	----	---

25	10	72
----	----	----

18	40
----	----

11	64	58
----	----	----

32	9
----	---

25	10	25	72
----	----	----	----

18	40
----	----

11	64	58
----	----	----

9	32
---	----

9	10	11	18	25	32	40	58	64	72
---	----	----	----	----	----	----	----	----	----

→ Quick sort:-

Pivot = 25

→ [10, 18, 11, 9] | 25 | [72, 40, 64, 58, 32]

Pivot = 10

→ [9] | 10 | [18, 11, 72, 40]

Pivot = 18

→ [11] | 18

Pivot = 72
 $\rightarrow [40, 64, 58, 32] | 72 \rightarrow [32] | 40 | [64, 58]$

Pivot = 64
 $\rightarrow [58] | 64$

Final: $[9, 10, 11, 18, 25, 32, 40, 58, 64, 72]$

5 > 80, 60, 70, 40, 10, 30, 50, 20

80 | 60 | 70 | 40 | 10 | 30 | 50 | 20

80 | 60 | 70

10 | 30 | 50 | 20

80 | 60

70 | 70

10 | 30

50 | 20

10 | 20 | 30 | 40 | 50 | 60 | 70 | 80 | 90

Quick Sort:-

Pivot = 80

$\rightarrow [60, 70, 70, 10, 30, 50, 80] | 80 \rightarrow [60, 70, 70] | [10, 30, 50]$

Pivot = 60

$\rightarrow [40, 10, 30, 50, 20] | 60 | 70$

Pivot = 40

$\rightarrow [10, 30, 20] | 40 | [50]$

Pivot = 10

$\rightarrow [] | 10 | [30, 20]$

Pivot = 30

$\rightarrow [20] | 30$

Final:

$[10, 20, 30, 40, 50, 60, 70, 80]$

6 > 0, U, 1, C, K, S, D, R, T

Merge sort:- $[0, U, 1, C] [K, S, D, R, T]$

$\rightarrow [0, U] [1, C] | [K, S] [D, R, T]$

$\rightarrow [0] [U] [1] [C] [K] [S] [D] [R] [T]$

Quick sort:-

Pivot = 0

$\rightarrow [1, C, K, D] | 0 | [U, S, R, T]$

Pivot = 1

$\rightarrow [C] | 1 | [K, D]$

Pivot = K

$\rightarrow [K] | [0]$

Finals $[C, 1, K, D, 0, R, S, T, U]$

7 > M, E, R, G, E, S, O, R, T

Merge sort:- $[M, E, R, G] [E, S, O, R, T]$

$\rightarrow [M, E], [R, G] | [E, S], [O, R, T]$

$\rightarrow [E, M], [G, R] \rightarrow [E, M, G, R]$

$\rightarrow [E, S], [O, R, T] \rightarrow [E, O, R, S, T]$

Finals:- $[E, E, G, M, O, R, R, S, T]$

Quick sort:-

Pivot = M

Right side:-

Pivot = U

$\rightarrow [S, R, T] | U$

Pivot = S

$\rightarrow [R] | S | [T]$

$\rightarrow [E, G, E] | M | [R, S, D, R, T]$

Pivot = E

$\rightarrow E | [G, E]$

Pivot = G

$\rightarrow [E] | G$

Pivot = R

$\rightarrow [D] | R | [S, R, T]$

Pivot = S

$\rightarrow [R] | S | [T]$

Final: $[E, E, G, M, D, R, R, S, T]$